



Visvesvaraya Technological University



(State University of Government of Karnataka as per the VTU Act ,1994) "Jnana Sangama" Belagavi – 590018 ,Karnataka ,India.



**Department of Computer Science and Engineering
“Jnana Sangama”, VTU-Campus, Belagavi-590018**

LABORATORY JOURNAL

Academic Year : 2026-27

Name :
USN :
Semester : 7th sem
Subject : IOT Lab Manual
Subject Code : BCS701
Program Name : Computer Science & Engineering



Visvesvaraya Technological University

(State University of Government of Karnataka as per the VTU Act,1994) "Jnana Sangama" Belagavi – 590018 ,Karnataka ,India.



Department of Computer Science and Engineering Bachelor of Technology (B.Tech.)

Computer Science and Engineering

This is to certify that Mr/Ms _____ with USN _____ has satisfactorily completed all laboratory assignments of the subject **IOT Lab** with subject code **BCS701** during the academic year 2026 - 2027.

Faculty in-charge

Mr. Chinmay Prabhu

Chairperson

Prof.S A Angadi

Name of the examiner

1. _____

2. _____

Signature of the examiner with date

INDEX

S.NO	EXPERIMENT NAME	Pg.No	DATE	Sign
1.	Develop a program to blink 5 LEDs back and forth.			
2.	Develop a program to interface a relay with Arduino board.			
3.	Develop a program to deploy an intrusion detection system using Ultrasonic and sound sensors			
4.	Develop a program to control a DC motor with Arduino board.			
5.	Develop a program to deploy smart street light system using LDR sensor.			
6.	Develop a program to classify dry and wet waste with the Moisture sensor (DHT22).			
7.	Develop a program to read the pH value of various substances like milk, lime and water.			
8.	Develop a program to detect the gas leakage in the surrounding environment.			
9.	Develop a program to demonstrate weather station readings using Arduino.			
10.	Develop a program to setup a UART protocol and pass a string through the protocol.			
11.	Develop a water level depth detection system using Ultrasonic sensor			
12.	Develop a program to simulate interfacing with the keypad module to record the keystrokes.			

Virtual Laboratory Setup and Execution Guide

1. Introduction

This virtual laboratory environment enables students to perform Internet of Things (IoT) experiments using Arduino UNO, Visual Studio Code (VS Code), PlatformIO IDE, and the Wokwi simulator. It provides a software-based environment for writing, compiling, testing, and simulating embedded system programs without requiring all the physical hardware components.

The laboratory setup helps students understand Arduino programming, sensor interfacing, digital and analog input/output, serial communication, and basic IoT concepts through virtual experiments.

2. Software Requirements and Initial Setup

Install and configure the following software and extensions before performing the experiments.

1. Visual Studio Code: Download and install Visual Studio Code from the official website: <https://code.visualstudio.com/>
2. PlatformIO IDE: Open VS Code, navigate to the Extensions panel, search for *PlatformIO IDE*, and install the extension. PlatformIO is used to manage the Arduino project, compile the source code, and generate firmware.
3. Wokwi for VS Code: Search for *Wokwi for VS Code* in the Extensions panel and install it. Wokwi provides a virtual environment for simulating supported electronic components and microcontroller circuits.
4. Wokwi License Activation: Open the VS Code Command Palette using Ctrl + Shift + P and search for Wokwi: Request a new License. Follow the instructions displayed to complete activation, if required by the installed extension and account.
5. Internet Connection: An internet connection is generally required for initial toolchain installation, downloading dependencies, Wokwi license activation, and online simulation services.

3. Procedure to Execute an Experiment

Follow these steps to build and simulate any experiment.

1. Open Visual Studio Code.
2. Select File → Open Folder and open the required experiment folder, such as Experiment_01.
3. Allow PlatformIO to initialize the project and download the required toolchain and libraries when prompted.
4. Compile the program using either of the following methods:
 - Click the PlatformIO Build (✓) button in the VS Code interface.
 - Open the integrated terminal in the project folder and execute pio run, provided the PlatformIO command is available in the terminal.
5. Verify that the compilation completes successfully without errors.
6. Open the Command Palette using Ctrl + Shift + P.
7. Select Wokwi: Start Simulator to launch the virtual circuit, provided the project and simulator configuration are valid.
8. Interact with the virtual components, such as switches, LEDs, sensors, or potentiometers, according to the experiment.
9. Observe the circuit behaviour and check the Serial Monitor output wherever required.
10. Record the observations and results in the laboratory record.

4. Important Files in Each Experiment

Each experiment folder contains the following important files.

File Name	Description
src/main.cpp	Contains the Arduino program written in C++.
platformio.ini	Specifies the PlatformIO environment, target board, framework, and required configuration.
diagram.json	Defines the virtual circuit components and their connections in Wokwi.

wokwi.toml	Configures the Wokwi simulator to locate the compiled firmware and the required simulation files.
README.md	Provides the experiment description, execution procedure, and student activities

5. Firmware Generation

When an experiment is compiled successfully, PlatformIO generates the firmware files in the project's `.pio/build/uno/` directory, provided that the environment is configured with the name `uno`.

The main firmware files are:

- `.pio/build/uno/firmware.hex` – Intel HEX firmware file.
- `.pio/build/uno/firmware.elf` – ELF executable containing program and debugging information.

The `wokwi.toml` configuration must point to the appropriate firmware file and any required build configuration. The firmware paths should match the actual project configuration.

6. Troubleshooting

Problem: The `pio` command is not recognized.

Solution: Use the PlatformIO Build button in VS Code. Alternatively, open a PlatformIO-enabled terminal or configure the PlatformIO Core executable in the system PATH.

Problem: The simulator does not start.

Solution: Verify that the Wokwi extension is installed and activated, the project contains a valid `diagram.json` and `wokwi.toml`, and the firmware has been compiled successfully.

Problem: Compilation errors occur.

Solution: Check the source code, board configuration in `platformio.ini`, required libraries, and compiler error messages.

Problem: Components do not respond as expected.

Solution: Verify the pin connections in `diagram.json`, ensure that the pins used in the source code match the circuit, and check the program logic.

Experiment No:1 Develop a program to blink 5 LEDs back and forth

Aim: To develop an Arduino program that blinks five LEDs back and forth (left to right and right to left) in a continuous chasing pattern

Components Required:

Particulars	Quantity
Arduino Uno	1
LED	5
Resistor 220 ohm	5
Jumper Wires	As required

Program:

```
#include <Arduino.h>

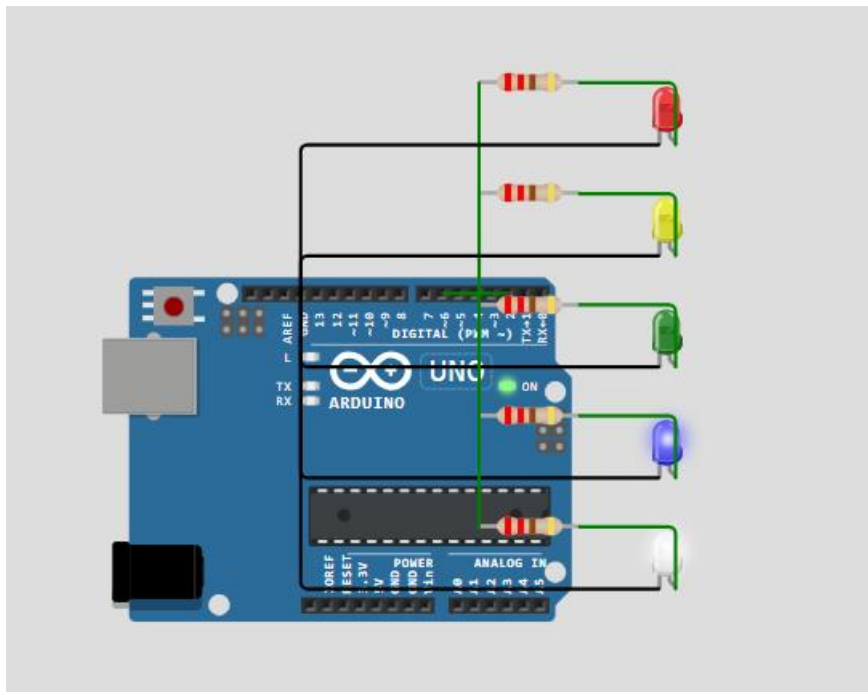
const byte leds[] = {2, 3, 4, 5, 6};
const byte LED_COUNT = 5;
const unsigned int STEP_DELAY = 500;

void setup() {
  for (byte i = 0; i < LED_COUNT; i++) {
    pinMode(leds[i], OUTPUT);
    digitalWrite(leds[i], LOW);
  }
}

void glowOne(byte index) {
  digitalWrite(leds[index], HIGH);
  delay(STEP_DELAY);
```

```
digitalWrite(leds[index], LOW);  
}  
  
void loop() {  
  for (byte i = 0; i < LED_COUNT; i++) {  
    glowOne(i);  
  }  
  for (int i = LED_COUNT - 2; i > 0; i--) {  
    glowOne(i);  
  }  
}
```

Simulation Evidence:



Observation Table:

Sl. No.	LED 2	LED 3	LED 4	LED 5	LED 6	Observation
1	ON	OFF	OFF	OFF	OFF	LED 1 glows
2	OFF	ON	OFF	OFF	OFF	LED 2 glows
3	OFF	OFF	ON	OFF	OFF	LED 3 glows
4	OFF	OFF	OFF	ON	OFF	LED 4 glows
5	OFF	OFF	OFF	OFF	ON	LED 5 glows
6	OFF	OFF	OFF	ON	OFF	LED 4 glows
7	OFF	OFF	ON	OFF	OFF	LED 3 glows
8	OFF	ON	OFF	OFF	OFF	LED 2 glows
9	OFF	OFF	OFF	OFF	OFF	Sequence repeats

Modification Performed :

1. **Added 5 LEDs** to the Wokwi Arduino circuit.
2. Connected the LEDs to **digital pins 2, 3, 4, 5, and 6**.
3. Connected a **220Ω resistor** in series with each LED.
4. Connected the LED cathodes to **GND**.
5. Modified the Arduino program to control all 5 LEDs using an **array**.
6. Added a glowOne() function to make each LED glow individually.
7. Set the LED glow duration to **500 ms** using STEP_DELAY.
8. Programmed the LEDs to glow **from left to right and then back toward the center**, creating a running LED effect.
9. Simulated the complete circuit in **Wokwi** to verify the LED sequence.

Result : The five LEDs glow sequentially, producing a running/chasing LED effect.

Experiment No: 2 Develop a program to interface a relay with Arduino board.

Aim: To develop an Arduino program to interface a relay with Arduino board.

Components Required:

Particulars	Quantity
Arduino Uno	1
Relay Module	1
DC Motor 9/12V	2
Power Supply 9/12V	1
Push Button	1
Jumper Wires	As required

Program:

```
#include <Arduino.h>

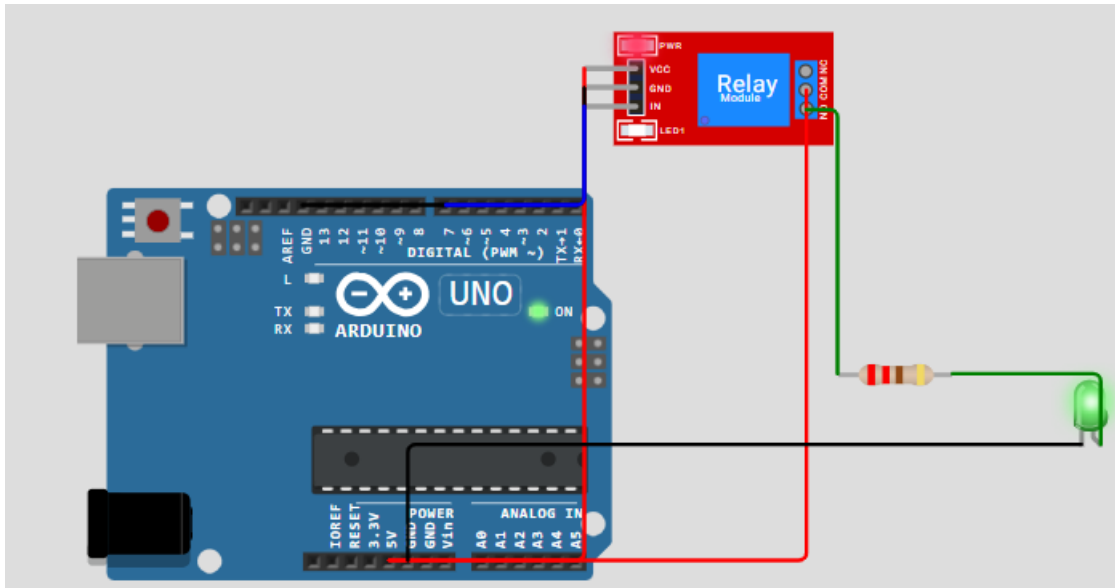
const byte RELAY_PIN = 7;

void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(RELAY_PIN, HIGH);
  Serial.println("Relay ON - Load ON");
  delay(2000);

  digitalWrite(RELAY_PIN, LOW);
  Serial.println("Relay OFF - Load OFF");
```

```
delay(2000);  
}
```

Simulation Evidence:**Observation Table:**

Sl. No.	Relay Pin (7)	Relay Status	Load Status	Time Delay	Serial Monitor Output
1	HIGH	ON	ON	2 seconds	Relay ON - Load ON
2	LOW	OFF	OFF	2 seconds	Relay OFF - Load OFF
3	HIGH	ON	ON	2 seconds	Relay ON - Load ON
4	LOW	OFF	OFF	2 seconds	Relay OFF - Load OFF

Modification Performed:

1. Added a relay module to the Wokwi circuit.
2. Connected the relay control pin to digital pin 7 of the Arduino.
3. Defined RELAY_PIN = 7 in the program.
4. Configured pin 7 as an OUTPUT.
5. Initially set the relay to LOW (OFF).
6. Added Serial Monitor communication at 9600 baud.
7. Added messages to display the relay and load status.
8. Modified the program to switch the relay ON and OFF every 2 seconds.
9. Simulated and tested the circuit using VS Code with Wokwi.

Result: The relay was successfully controlled using Arduino in Wokwi. The relay turns ON for 2 seconds and OFF for 2 seconds repeatedly, and the corresponding status messages are displayed on the Serial Monitor.

Experiment No: 3 Develop a program to deploy an intrusion detection system using Ultrasonic and sound sensors.

Aim: To develop an Arduino-based program to detect intrusion using an ultrasonic sensor (distance measurement) and a sound sensor (noise detection), and trigger an alert using buzzer/LED.

Components Required:

Particulars	Quantity
Arduino Uno	1
Ultrasonic Sensor	1
LED	1
Buzzer	1
Jumper Wires	As required

Program :

```
#include <Arduino.h>

const byte TRIG_PIN = 9;
const byte ECHO_PIN = 10;
const byte SOUND_SIM_PIN = A0;
const byte ALARM_LED = 6;
const float DISTANCE_THRESHOLD_CM = 50.0;
const int SOUND_THRESHOLD = 700;

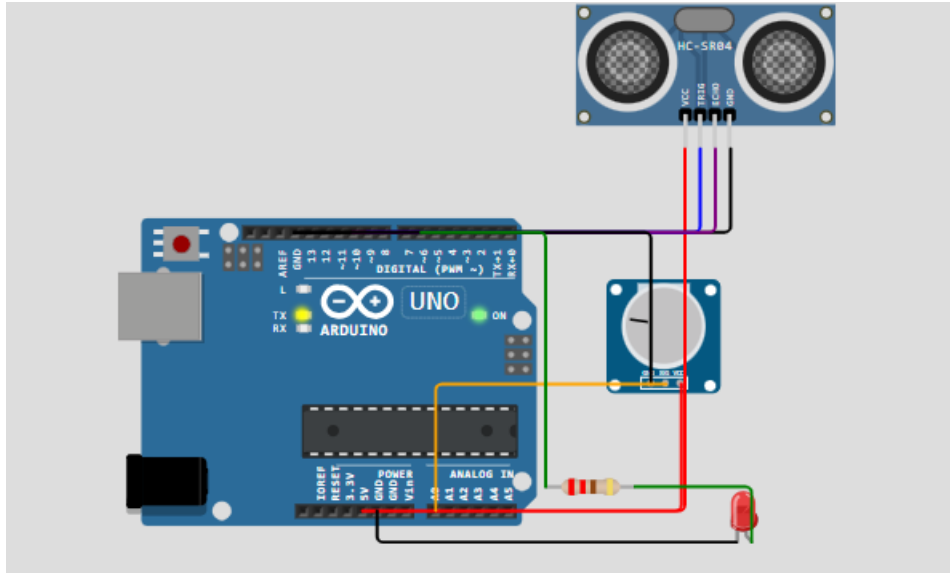
float readDistanceCm() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
```

```
    unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL);
    if (duration == 0) return -1.0;
    return duration / 58.0;
}

void setup() {
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(ALARM_LED, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    float distance = readDistanceCm();
    int soundLevel = analogRead(SOUND_SIM_PIN);
    bool objectNear = distance > 0 && distance < DISTANCE_THRESHOLD_CM;
    bool soundDetected = soundLevel > SOUND_THRESHOLD;
    bool intrusion = objectNear || soundDetected;
    digitalWrite(ALARM_LED, intrusion ? HIGH : LOW);
    Serial.print("Distance: ");
    if (distance < 0) Serial.print("No echo");
    else Serial.print(distance, 1);
    Serial.print(" cm | Sound level: ");
    Serial.print(soundLevel);
    Serial.print(" | Intrusion: ");
    Serial.println(intrusion ? "YES" : "NO");
    delay(250);
}
```

Simulation Evidence:



Observation Table:

Sl. No.	Distance (cm)	Sound Level	Object Near?	Sound Detected?	Intrusion	Alarm LED
1	80	300	NO	NO	NO	OFF
2	60	500	NO	NO	NO	OFF
3	40	400	YES	NO	YES	ON
4	30	800	YES	YES	YES	ON
5	70	800	NO	YES	YES	ON
6	90	300	NO	NO	NO	OFF

Modification Performed:

1. Added an HC-SR04 ultrasonic sensor to measure object distance.
2. Connected TRIG to pin 9 and ECHO to pin 10.
3. Added an analog sound sensor/simulation input to A0.
4. Added an alarm LED on digital pin 6.
5. Implemented the readDistanceCm() function to calculate distance using the ultrasonic sensor.
6. Added a 50 cm distance threshold for detecting nearby objects.
7. Added a 700 sound-level threshold for detecting abnormal sound.
8. Modified the program to trigger the alarm when either an object is nearby or excessive sound is detected.
9. Added Serial Monitor output to display distance, sound level, and intrusion status.
10. Set the monitoring interval to 250 ms for continuous detection.

Result: The intrusion detection system was successfully implemented in simulated using Wokwi. The ultrasonic sensor detects nearby objects and the sound sensor detects high sound levels. When either condition is satisfied, the alarm LED turns ON and the Serial Monitor displays Intrusion: YES; otherwise, the LED remains OFF.

Experiment No: 4 Develop a program to control a DC motor with Arduino board

Aim: To develop an Arduino program to control the direction of a DC motor using an Arduino board and an L293D motor driver.

Components Required:

Particulars	Quantity
Arduino Uno	1
DC Motor	1
Motor Driver Module	1
Jumper Wires	As needed

Program:

```
#include <Arduino.h>

const byte EN_PWM = 5;

const byte IN1 = 8;

const byte IN2 = 9;

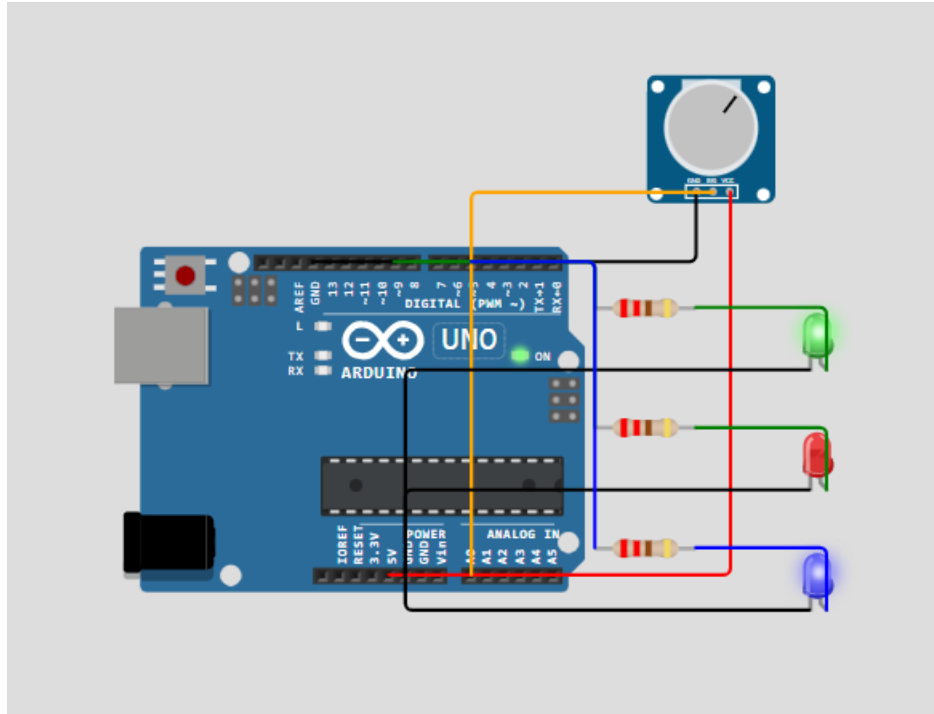
const byte SPEED_POT = A0;

void setMotor(bool forward, byte speedValue) {
    digitalWrite(IN1, forward ? HIGH : LOW);
    digitalWrite(IN2, forward ? LOW : HIGH);
    analogWrite(EN_PWM, speedValue);
}

void stopMotor() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    analogWrite(EN_PWM, 0);
}
```

```
void setup() {  
  pinMode(EN_PWM, OUTPUT);  
  pinMode(IN1, OUTPUT);  
  pinMode(IN2, OUTPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  int raw = analogRead(SPEED_POT);  
  byte speedValue = map(raw, 0, 1023, 0, 255);  
  setMotor(true, speedValue);  
  Serial.print("FORWARD | PWM speed = ");  
  Serial.println(speedValue);  
  delay(3000);  
  stopMotor();  
  Serial.println("STOP");  
  delay(1000);  
  raw = analogRead(SPEED_POT);  
  speedValue = map(raw, 0, 1023, 0, 255);  
  setMotor(false, speedValue);  
  Serial.print("REVERSE | PWM speed = ");  
  Serial.println(speedValue);  
  delay(3000);  
  stopMotor();  
  Serial.println("STOP");  
  delay(1000);  
}
```

Simulation Evidence:



Observation Table:

Sl. No.	Potentiometer Value	PWM Speed	Motor Direction	Motor Status	Duration
1	0	0	Forward	STOP	3 sec
2	512	127	Forward	ON	3 sec
3	1023	255	Forward	ON	3 sec
4	—	0	—	STOP	1 sec
5	512	127	Reverse	ON	3 sec
6	1023	255	Reverse	ON	3 sec
7	—	0	—	STOP	1 sec

Modification Performed:

1. Added a **DC motor** and motor driver/control circuit in Wokwi.
2. Connected the **PWM/Enable pin to digital pin 5**.
3. Connected motor direction control pins **IN1 to pin 8** and **IN2 to pin 9**.
4. Added a **potentiometer at A0** to control motor speed.
5. Used `analogRead()` to read the potentiometer value from **0–1023**.
6. Used `map()` to convert the potentiometer value into a PWM range of **0–255**.
7. Created the `setMotor()` function for controlling **direction and speed**.
8. Created the `stopMotor()` function to stop the motor.
9. Modified the program to rotate the motor **forward and reverse automatically**.
10. Added **Serial Monitor messages** to display direction and PWM speed.

Result: The DC motor was successfully controlled using Wokwi. The potentiometer controls the motor speed through PWM, while the motor automatically rotates in the forward direction for 3 seconds, stops, rotates in reverse for 3 seconds, and repeats the sequence.

Experiment No: 5 Develop a program to deploy smart street light system using LDR sensor.

Aim: To develop an Arduino program that automatically turns street lights ON at night and OFF during daylight using an LDR (light-dependent resistor) as the light sensor.

Components Required:

Particulars	Quantity
Arduino Uno	1
LDR/Photo Sensor	1
LED	1
10K-ohm resistor	1
Jumper Wires	7

Program:

```
#include <Arduino.h>

const byte LDR_PIN = A0;
const byte STREET_LIGHT = 6;
const int DARK_THRESHOLD = 600;

void setup() {
  pinMode(STREET_LIGHT, OUTPUT);
  Serial.begin(9600);
}

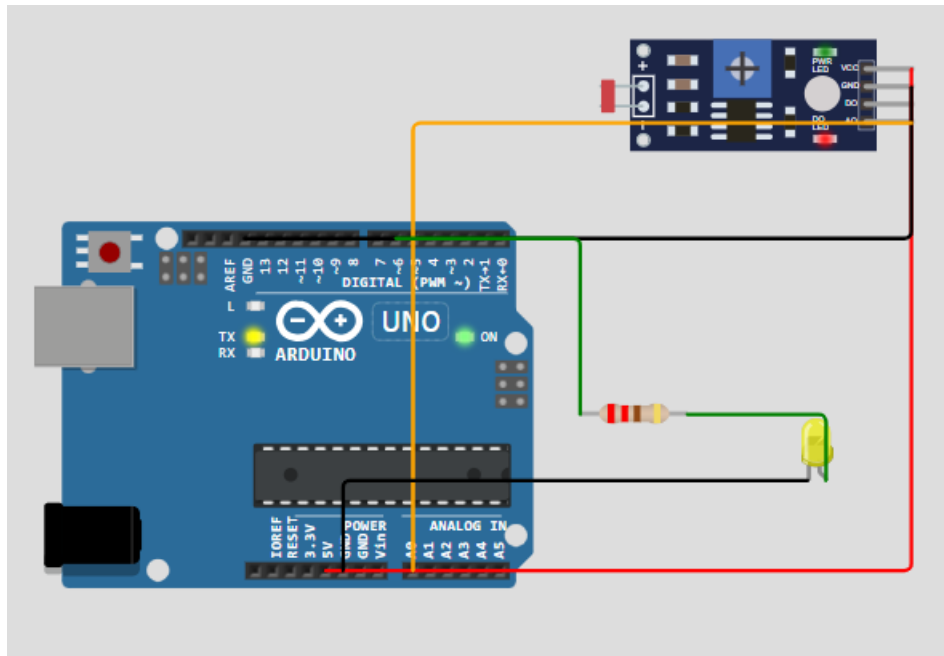
void loop() {
  int ldrValue = analogRead(LDR_PIN);
  bool isDark = ldrValue > DARK_THRESHOLD;
  digitalWrite(STREET_LIGHT, isDark ? HIGH : LOW);
  Serial.print("LDR value: ");
  Serial.print(ldrValue);
  Serial.print(" | Street light: ");
```

```

Serial.println(isDark ? "ON" : "OFF");
delay(300);
}

```

Simulation Evidence:



Observation Table:

Sl. No.	LDR Value	Light Condition	Dark Detected	Street Light
1	200	Bright	NO	OFF
2	400	Bright	NO	OFF
3	550	Normal Light	NO	OFF
4	650	Dark	YES	ON
5	800	Dark	YES	ON
6	1000	Very Dark	YES	ON

Modification Performed:

1. Added an **LDR/light sensor** to the Wokwi circuit.
2. Connected the LDR output to **analog pin A0**.
3. Added an **LED to digital pin 6** to represent the street light.
4. Set **600 as the dark threshold value**.
5. Used `analogRead()` to continuously read the LDR value.
6. Added a condition to detect whether the environment is dark.
7. The street light automatically turns **ON in darkness** and **OFF in sufficient light**.
8. Added **Serial Monitor output** to display the LDR value and street-light status.
9. Added a **300 ms delay** for continuous monitoring.

Result: The automatic street light system was successfully implemented and simulated in Wokwi. The LDR detects the surrounding light intensity, and the street light automatically turns ON when the LDR value exceeds 600 (dark condition) and turns OFF when the value is 600 or below (bright condition).

Experiment No: 6 Develop a program to classify dry and wet waste with the Moisture sensor (DHT22).

Aim: To develop an Arduino program that classifies dry waste and wet waste using the DHT22 humidity sensor, based on the moisture level detected.

Components Required:

Particulars	Quantity
Arduino Uno	1
DHT22 Sensor Module	2
Jumper Wires	As required

Program:

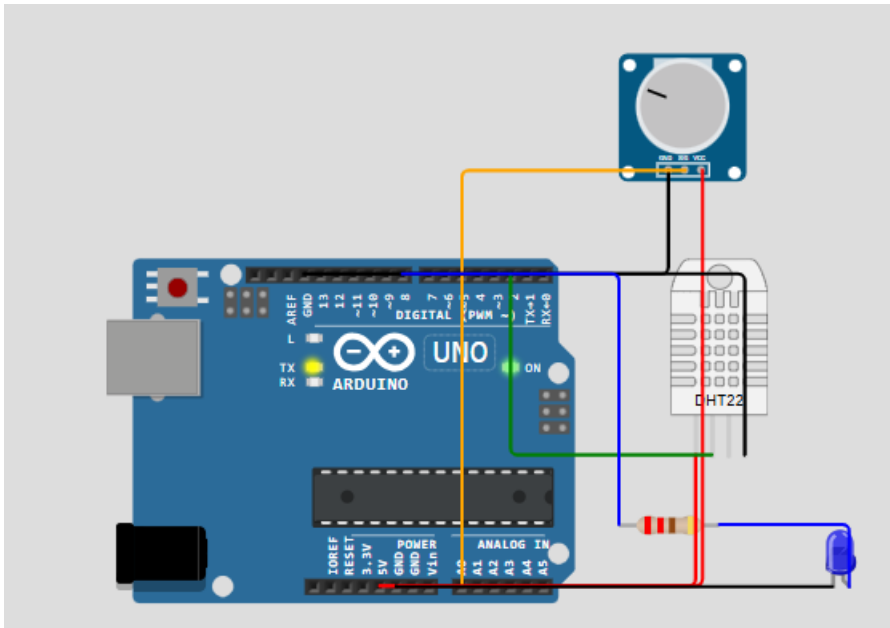
```
#include <Arduino.h>
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT22

const byte MOISTURE_SIM_PIN = A0;
const byte WET_LED = 8;
const int WET_THRESHOLD = 500;

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  pinMode(WET_LED, OUTPUT);
```

```
dht.begin();  
  
}  
  
void loop() {  
  int moistureSignal = analogRead(MOISTURE_SIM_PIN);  
  float humidity = dht.readHumidity();  
  float temperature = dht.readTemperature();  
  
  bool wetWaste = moistureSignal >= WET_THRESHOLD;  
  digitalWrite(WET_LED, wetWaste ? HIGH : LOW);  
  
  Serial.print("Moisture simulation: ");  
  Serial.print(moistureSignal);  
  Serial.print(" | Classification: ");  
  Serial.print(wetWaste ? "WET WASTE" : "DRY WASTE");  
  
  if (!isnan(humidity) && !isnan(temperature)) {  
    Serial.print(" | Ambient T=");  
    Serial.print(temperature, 1);  
    Serial.print(" C, RH=");  
    Serial.print(humidity, 1);  
    Serial.print("%");  
  }  
  Serial.println();  
  
  delay(1000);  
}
```

Simulation Evidence:**Observation Table:**

Sl. No.	Moisture Value	Humidity (%)	Temperature (°C)	Classification	Wet LED
1	200	45	25.0	DRY WASTE	OFF
2	350	50	26.0	DRY WASTE	OFF
3	499	55	27.0	DRY WASTE	OFF
4	500	60	27.5	WET WASTE	ON
5	650	70	28.0	WET WASTE	ON
6	800	80	29.0	WET WASTE	ON

Modification Performed:

1. Added a DHT22 temperature and humidity sensor to the Wokwi circuit.
2. Connected the DHT22 data pin to digital pin 2.
3. Added a simulated moisture sensor connected to analog pin A0.
4. Added an LED to digital pin 8 to indicate wet waste.
5. Set the moisture threshold to 500.
6. Used `analogRead()` to obtain the moisture sensor value.
7. Used the DHT22 sensor to measure temperature and humidity.
8. Added a classification condition:
 - Moisture $\geq$ 500 $\rightarrow$ WET WASTE
 - Moisture $<$ 500 $\rightarrow$ DRY WASTE
9. Added Serial Monitor output to display moisture, classification, temperature, and humidity.
10. Added a 1-second delay for continuous monitoring.

Result: The wet and dry waste classification system was successfully implemented and simulated in Wokwi. The moisture value is used to classify waste as wet or dry, while the DHT22 sensor displays the surrounding temperature and humidity. The LED turns ON when wet waste is detected and remains OFF for dry waste.

Experiment No: 7 Develop a program to read the pH value of a various substances like milk, lime and water.

Aim: To develop an Arduino program to measure the pH level of different substances such as milk, lime juice, and water using an analog pH sensor.

Components Required:

Particulars	Quantity
Arduino Uno	1
Bluetooth Module	1
LED	1
Jumper Wires	-

Program :

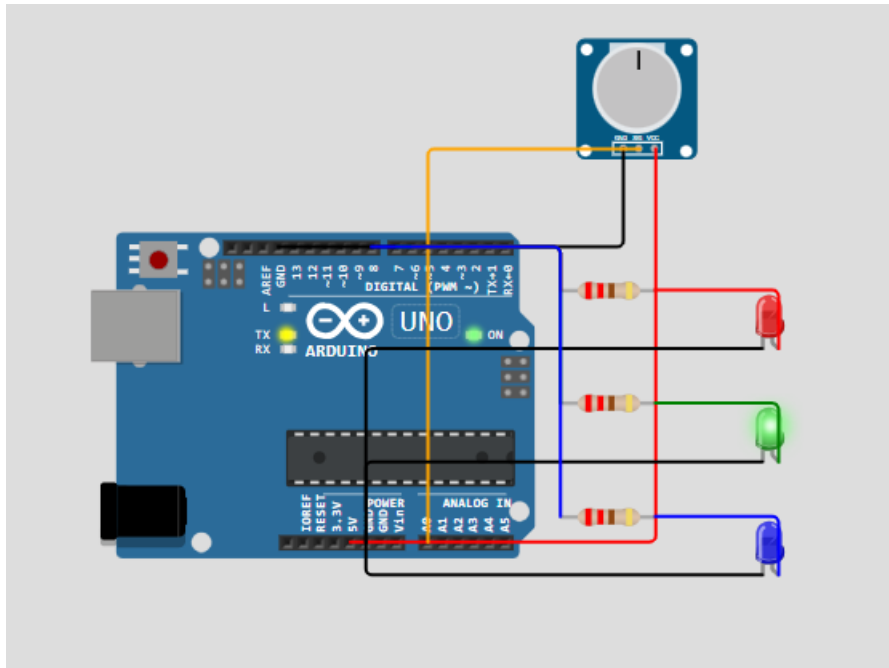
```
#include <Arduino.h>

const byte PH_SIM_PIN = A0;
const byte ACID_LED = 6;
const byte NEUTRAL_LED = 7;
const byte ALKALINE_LED = 8;

void setup() {
  pinMode(ACID_LED, OUTPUT);
  pinMode(NEUTRAL_LED, OUTPUT);
  pinMode(ALKALINE_LED, OUTPUT);
  Serial.begin(9600);
}
```

```
void loop() {  
  int raw = analogRead(PH_SIM_PIN);  
  float pH = raw * 14.0 / 1023.0;  
  digitalWrite(ACID_LED, LOW);  
  digitalWrite(NEUTRAL_LED, LOW);  
  digitalWrite(ALKALINE_LED, LOW);  
  const char* category;  
  if (pH < 6.5) {  
    category = "ACIDIC";  
    digitalWrite(ACID_LED, HIGH);  
  } else if (pH <= 7.5) {  
    category = "NEUTRAL";  
    digitalWrite(NEUTRAL_LED, HIGH);  
  } else {  
    category = "ALKALINE";  
    digitalWrite(ALKALINE_LED, HIGH);  
  }  
  Serial.print("Simulated pH = ");  
  Serial.print(pH, 2);  
  Serial.print(" | ");  
  Serial.println(category);  
  delay(500);  
}
```

Simulation Evidence :



Observation Table:

Sl. No.	Analog Value	Simulated pH	Category	Acid LED (Pin6)	Neutral LED (Pin 7)	Alkaline LED (Pin8)
1	100	1.37	ACIDIC	ON	OFF	OFF
2	400	5.47	ACIDIC	ON	OFF	OFF
3	475	6.50	NEUTRAL	OFF	ON	OFF
4	512	7.00	NEUTRAL	OFF	ON	OFF
5	548	7.50	NEUTRAL	OFF	ON	OFF
6	700	9.58	ALKALINE	OFF	OFF	ON
7	900	12.31	ALKALINE	OFF	OFF	ON

Modification Performed:

1. Added a potentiometer/analog input in Wokwi to simulate the pH sensor.
2. Connected the simulated pH sensor to analog pin A0.
3. Added three LEDs for pH classification:
 - Pin 6 → Acidic LED
 - Pin 7 → Neutral LED
 - Pin 8 → Alkaline LED
4. Converted the analog sensor value from 0–1023 into a pH range of 0–14.
5. Added conditions to classify the solution as acidic, neutral, or alkaline.
6. Programmed the circuit so that only the corresponding LED glows.
7. Added Serial Monitor output to display the simulated pH value and category.
8. Added a 500 ms delay for continuous monitoring.

Result : The pH detection and classification system was successfully implemented and simulated in Wokwi. The analog input is converted into a pH value from 0 to 14. The system correctly identifies the solution as acidic, neutral, or alkaline and turns ON the corresponding LED.

Experiment No: 8 Develop a program to detect the gas leakage in the surrounding environment.

Aim : To develop an Arduino-based system to detect gas leakage in the surrounding environment using an MQ-series gas sensor and trigger an alarm using a buzzer and LED.

Components Required:

Particulars	Quantity
Arduino Uno	1
MQ2 Gas Sensor	2
LED	1
Buzzer	1
Jumper Wires	As required

Program:

```
#include <Arduino.h>

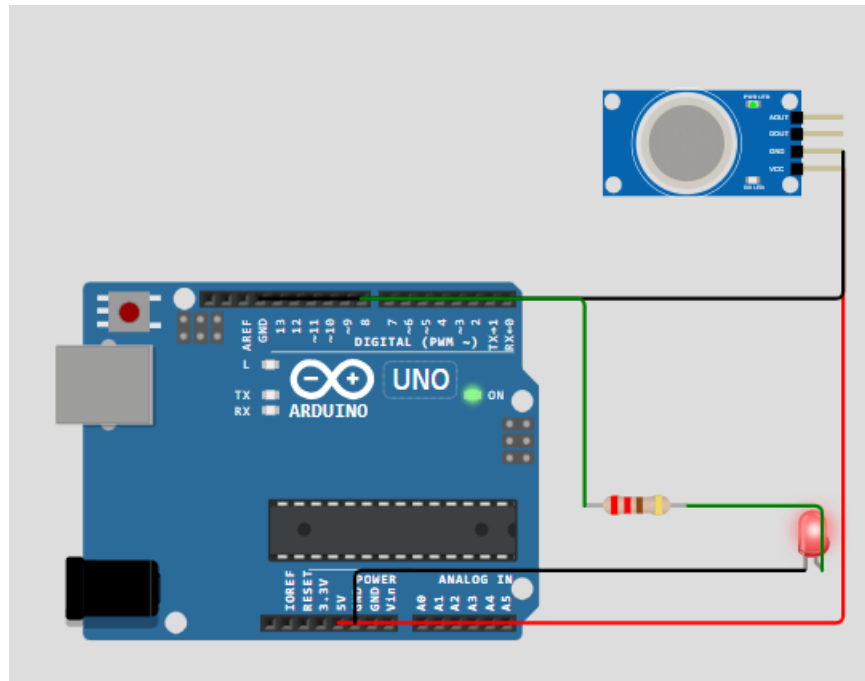
const byte GAS_ANALOG = A0;
const byte GAS_DIGITAL = 7;
const byte ALARM_LED = 8;

void setup() {
  pinMode(GAS_DIGITAL, INPUT);
  pinMode(ALARM_LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int gasRaw = analogRead(GAS_ANALOG);
  bool leakage = digitalRead(GAS_DIGITAL) == LOW;
  digitalWrite(ALARM_LED, leakage ? HIGH : LOW);
```

```
Serial.print("MQ2 analog value: ");  
Serial.print(gasRaw);  
Serial.print(" | Gas alarm: ");  
Serial.println(leakage ? "ON" : "OFF");  
  
delay(500);  
}
```

Simulation Evidence:



Observation Table:

Sl. No.	MQ-2 Analog Value	Digital Output (Pin 7)	Gas Leakage	Alarm LED (Pin 8)
1	120	HIGH	NO	OFF
2	250	HIGH	NO	OFF
3	400	HIGH	NO	OFF
4	550	LOW	YES	ON
5	700	LOW	YES	ON
6	900	LOW	YES	ON

Modification Performed:

1. Added an MQ-2 gas sensor to the Wokwi circuit.
2. Connected the MQ-2 analog output to A0.
3. Connected the digital output of the MQ-2 to digital pin 7.
4. Added an alarm LED to digital pin 8.
5. Configured pin 7 as an INPUT to read the gas sensor's digital signal.
6. Used `analogRead()` to monitor the analog gas value.
7. Used `digitalRead()` to detect gas leakage.
8. Programmed the alarm LED to turn ON when leakage is detected.
9. Added Serial Monitor output to display the MQ-2 analog value and gas alarm status.
10. Added a 500 ms delay for continuous monitoring.

Result: The gas leakage detection system was successfully implemented and simulated in VS Code using Wokwi. The MQ-2 sensor monitors the gas level, and when its digital output indicates leakage, the alarm LED turns ON. The analog gas value and alarm status are also displayed on the Serial Monitor.

Experiment No: 9 Develop a program to demonstrate weather station readings using Arduino.

Aim: To develop a program to demonstrate weather station readings using Arduino.

Components Required:

Particulars	Quantity
NodeMCU	1
DHT11 Sensor Module	2
Jumper Wires	-

Program :

```
#include <Arduino.h>
```

```
#include <DHT.h>
```

```
#define DHTPIN 2
```

```
#define DHTTYPE DHT22
```

```
const byte LDR_PIN = A0;
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  dht.begin();
```

```
}
```

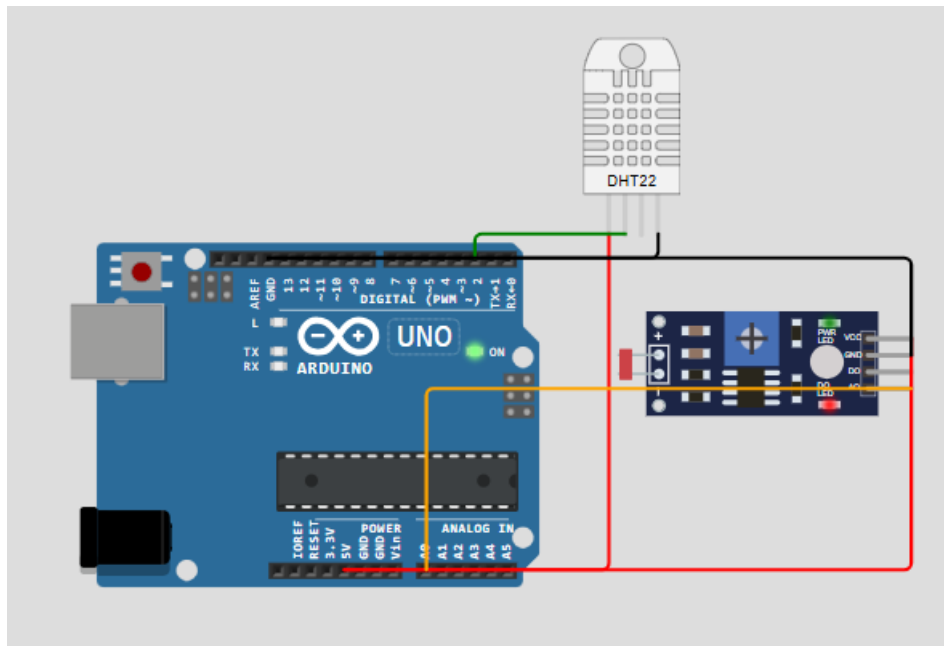
```
void loop() {
```

```
  float humidity = dht.readHumidity();
```

```
float temperature = dht.readTemperature();
int lightValue = analogRead(LDR_PIN);
if (isnan(humidity) || isnan(temperature)) {
    Serial.println("DHT22 read failed");
    delay(2000);
    return;
}
Serial.println("----- IOT WEATHER STATION -----");
Serial.print("Temperature: ");
Serial.print(temperature, 1);
Serial.println(" C");
Serial.print("Humidity: ");
Serial.print(humidity, 1);
Serial.println(" %");
Serial.print("Light ADC: ");
Serial.println(lightValue);
Serial.println();

delay(2000);
}
```

Simulation Evidence :



Observation Table :

Sl. No.	Temperature (°C)	Humidity (%)	LDR / Light ADC	Weather/Light Condition
1	22.5	45.0	850	Cool, bright
2	25.0	55.0	700	Normal, bright
3	28.5	65.0	500	Warm, moderate light
4	30.0	70.0	300	Hot, low light
5	26.5	60.0	150	Normal, dark

Modification Performed :

1. Added a DHT22 temperature and humidity sensor to the Wokwi circuit.
2. Connected the DHT22 data pin to digital pin 2.
3. Added an LDR/light sensor and connected its output to analog pin A0.
4. Used the DHT library to read temperature and humidity.
5. Used analogRead() to obtain the LDR light value.
6. Added an error-checking condition using isnan() to detect a failed DHT22 reading.
7. Added a formatted IoT Weather Station display in the Serial Monitor.
8. Programmed the system to display temperature, humidity, and light ADC values.
9. Added a 2-second delay between successive readings.

Result : The IoT Weather Station was successfully implemented and simulated in Wokwi. The DHT22 sensor successfully provides temperature and humidity readings, while the LDR provides the light intensity value. All sensor readings are displayed continuously on the Serial Monitor at 9600 baud.

Experiment No: 10 Develop a program to setup a UART protocol and pass a string through the protocol.

Aim: To develop a program that sets up UART (Universal Asynchronous Receiver/Transmitter) communication on Arduino and transmits a string through the serial protocol.

Component Required:

SI No	Components	Quantity
1	Arduino Uno	2
2	USB cable	1
3	Jumper wires	-
4	Breadboard	1

Program :

```
#include <Arduino.h>
```

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial uart(2, 3);
```

```
unsigned long lastSend = 0;
```

```
int messageNumber = 1;
```

```
void setup() {
```

```
  Serial.begin(9600); // USB Serial Monitor
```

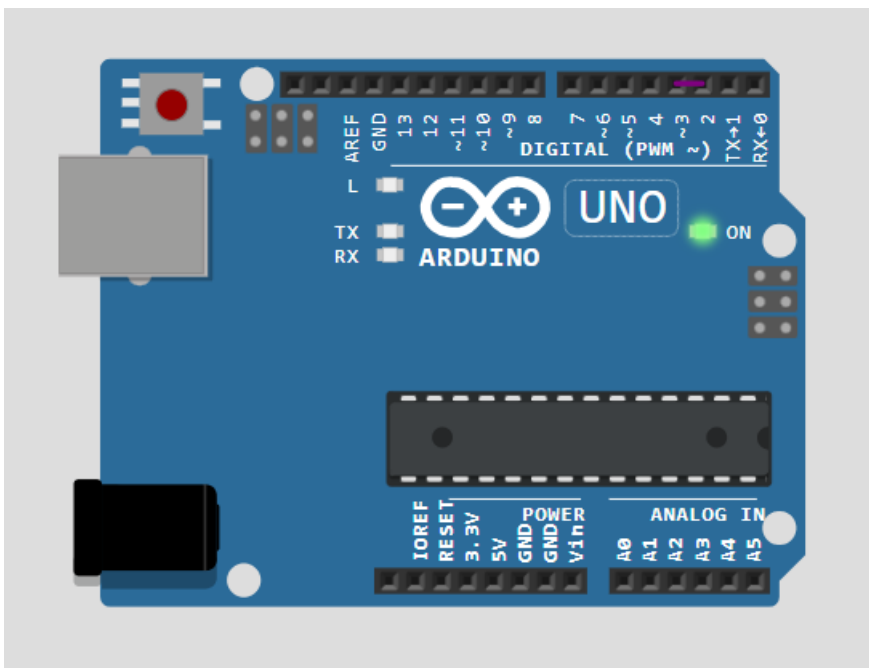
```
  uart.begin(9600); // Virtual UART link
```

```
  Serial.println("UART loopback test started.");
```

```
}
```

```
void loop() {  
  if (millis() - lastSend >= 2000) {  
    lastSend = millis();  
    uart.print("Hello IoT Students - Message ");  
    uart.println(messageNumber++);  
  
    delay(50);  
    Serial.print("Received through UART: ");  
    while (uart.available()) {  
      Serial.write(uart.read());  
    }  
  }  
}
```

Simulation Evidence :



Observation Table:

Sl. No.	Message	Received	Status
1	Message 1	Message 1	Success
2	Message 2	Message 2	Success
3	Message 3	Message 3	Success
4	Message 4	Message 4	Success
5	Message 5	Message 5	Success

Modification Performed:

1. Added the SoftwareSerial library for creating a virtual UART communication channel.
2. Configured pin 2 as RX and pin 3 as TX using SoftwareSerial uart(2, 3).
3. Set the virtual UART baud rate to 9600 baud.
4. Configured the USB Serial Monitor also at 9600 baud.
5. Modified the program to transmit a message every 2 seconds.
6. Added a message counter that automatically increments after every transmission.
7. Added a short 50 ms delay to allow the UART data to become available.
8. Used `uart.available()` to check whether data has been received.
9. Displayed the received UART data on the Serial Monitor.
10. Tested the UART communication using Wokwi simulation.

Result: The UART loopback communication was successfully implemented and simulated using Wokwi. Messages are transmitted through the virtual UART interface every 2 seconds and received data is displayed correctly on the Serial Monitor, confirming successful UART communication.

Experiment No: 11 Develop a water level depth detection system using Ultrasonic sensor..

Aim : To develop an Arduino-based system that measures the water depth (and level) inside a tank using an ultrasonic sensor and takes actions (display, buzzer/relay) based on measured depth.

Component Required:

SI No	Components	Quantity
1	Arduino uno	1
2	Ultrasonic sensor	1
3	LED	2
4	Jumper wire	--
5	Breadboard	1

Program :

```
#include <Arduino.h>
```

```
const byte TRIG_PIN = 9;
```

```
const byte ECHO_PIN = 10;
```

```
const float TANK_DEPTH_CM = 30.0;
```

```
float readDistanceCm() {
```

```
    digitalWrite(TRIG_PIN, LOW);
```

```
    delayMicroseconds(2);
```

```
    digitalWrite(TRIG_PIN, HIGH);
```

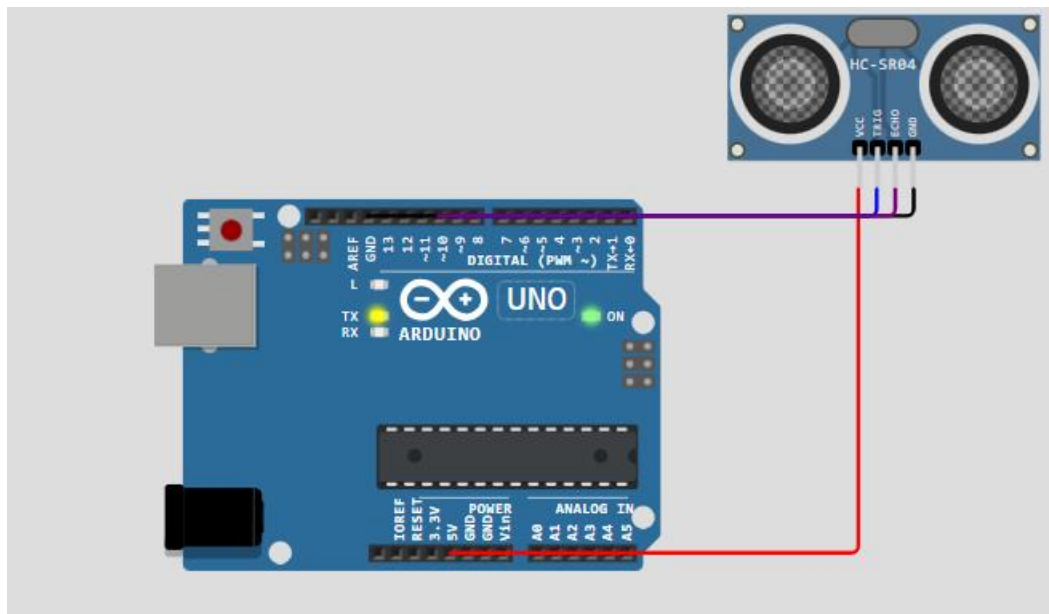
```
    delayMicroseconds(10);
```

```
    digitalWrite(TRIG_PIN, LOW);
```

```
unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL);  
if (duration == 0) return -1.0;  
return duration / 58.0;  
}  
  
void setup() {  
  pinMode(TRIG_PIN, OUTPUT);  
  pinMode(ECHO_PIN, INPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  float airGap = readDistanceCm();  
  if (airGap < 0) {  
    Serial.println("No valid echo");  
    delay(500);  
    return;  
  }  
  
  float waterDepth = TANK_DEPTH_CM - airGap;  
  if (waterDepth < 0) waterDepth = 0;  
  if (waterDepth > TANK_DEPTH_CM) waterDepth = TANK_DEPTH_CM;  
  
  float fillPercent = waterDepth * 100.0 / TANK_DEPTH_CM;  
  
  Serial.print("Air gap: ");
```

```
Serial.print(airGap, 1);  
Serial.print(" cm | Water depth: ");  
Serial.print(waterDepth, 1);  
Serial.print(" cm | Fill: ");  
Serial.print(fillPercent, 1);  
Serial.println("%");  
  
delay(500);  
}
```

Simulation Evidence :



Observation Table :

Sl. No.	Air Gap (cm)	Water Level (cm)	Fill (%)
1	25	5	16.7
2	20	10	33.3
3	15	15	50.0
4	10	20	66.7
5	5	25	83.3
6	2	28	93.3

Modification Performed :

1. Added an HC-SR04 ultrasonic sensor to measure the water level.
2. Connected TRIG to pin 9 and ECHO to pin 10.
3. Set the tank depth to 30 cm.
4. Calculated the water depth and fill percentage from the measured air gap.
5. Added Serial Monitor output for air gap, water depth, and fill percentage.
6. Added a check for invalid ultrasonic readings.

Result : The water tank level monitoring system was successfully simulated in Wokwi. The ultrasonic sensor measures the distance from the sensor to the water surface and calculates the water level and tank fill percentage accurately.

Experiment No: 12 Develop a program to simulate interfacing with the keypad module to record the keystrokes.

Aim : To develop and execute an Arduino program to interface a 4×4 keypad module and record the keystrokes entered by the user.

Component Required:

SI No	Components	Quantity
1	Arduino Uno	1
2	Ultrasonic sensor	1
3	LED	2
4	Jumper wires	--
5	Breadboard	1

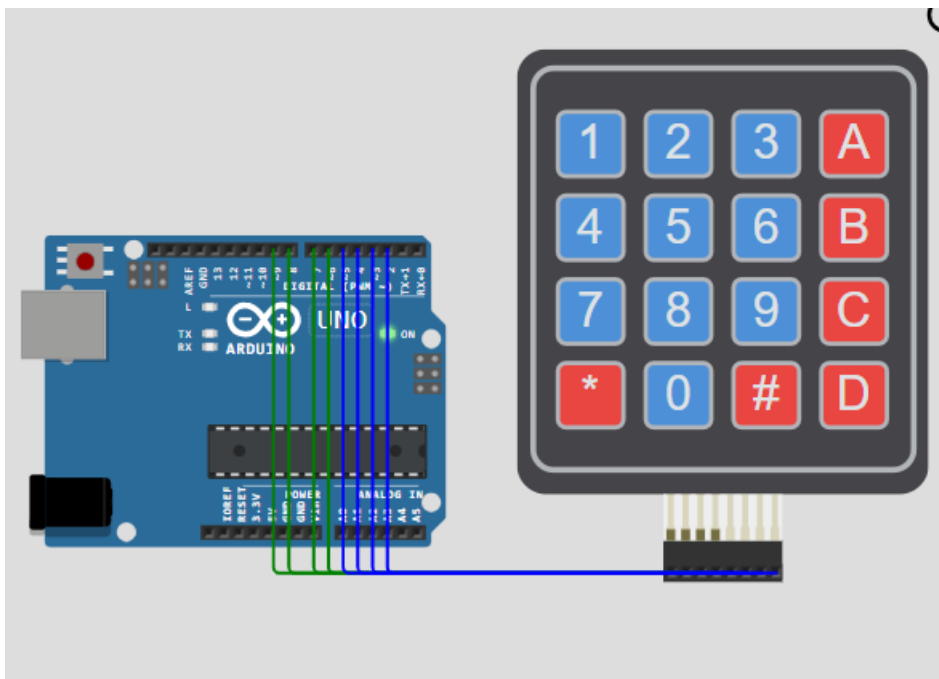
Program:

```
#include <Arduino.h>
#include <Keypad.h>
const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3, 2};
```

```
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
```

```
void setup() {  
  Serial.begin(9600);  
  Serial.println("Press keys on the 4x4 keypad:");  
}  
void loop() {  
  char key = keypad.getKey();  
  if (key) {  
    Serial.print("Key pressed: ");  
    Serial.println(key);  
  }  
}
```

Simulation Evidence :



Observation Table:

Sl. No.	Key Pressed	Serial Monitor Output
1	1	Key pressed: 1
2	5	Key pressed: 5
3	A	Key pressed: A
4	8	Key pressed: 8
5	*	Key pressed: *
6	#	Key pressed: #

Modification Performed:

1. Added a 4×4 keypad to the Wokwi circuit.
2. Connected the 4 row pins to Arduino pins 9, 8, 7, and 6.
3. Connected the 4 column pins to pins 5, 4, 3, and 2.
4. Used the Keypad library to interface the keypad.
5. Configured the keypad with numbers 0–9 and keys A–D, *, #.
6. Added Serial Monitor output to display the pressed key.

Result: The 4×4 keypad was successfully interfaced with Arduino in Wokwi. Whenever a key is pressed, the corresponding key value is displayed on the Serial Monitor.